

Cyber Security Project 1

Password Strength Checker

Industrial Training Documentation

```
password = input(...)
has_uppercase = any(...)
has_number = any(...)
has_symbol = any(...)
```

Weak | Medium | Strong



PASSWORD SECURITY

Student Name	Samaa Ahmad
Organization	DecodeLabs
Project	Project 1 - Password Strength Checker
Language	Python

Batch 2026

Contents

1. Introduction
2. Project Objective and Requirements
3. Password Strength Logic
4. IPO Model and Algorithm
5. Python Implementation
6. Code Explanation
7. Test Cases and Results
8. Requirement Validation
9. Security Notes and Limitations
10. Conclusion

This documentation is based on DecodeLabs Cyber Security Project 1 and the submitted Python implementation. The project focuses on string handling, conditional checks, and basic password-security logic.

1. Introduction

Passwords are one of the first defensive controls used to protect user accounts. A password-strength checker does not store or authenticate a password; instead, it evaluates whether the entered password satisfies a defined set of security characteristics. In this project, the program analyzes a password and classifies it as Weak, Medium, or Strong.

The DecodeLabs task specifically requires checking password length, uppercase letters, numbers, and symbols, then displaying the password-strength result. The submitted Python code implements these checks using built-in string methods and conditional logic.

2. Project Objective and Requirements

Objective: Create a Python program that checks whether a password is weak, medium, or strong.

Requirement	How it is implemented	Status
Check password length	<code>len(password) < 8</code>	Implemented
Check uppercase letters	<code>any(char.isupper() for char in password)</code>	Implemented
Check numbers	<code>any(char.isdigit() for char in password)</code>	Implemented
Check symbols	<code>any(not char.isalnum() for char in password)</code>	Implemented
Display strength result	<code>print("Password Strength:", strength)</code>	Implemented

3. Password Strength Logic

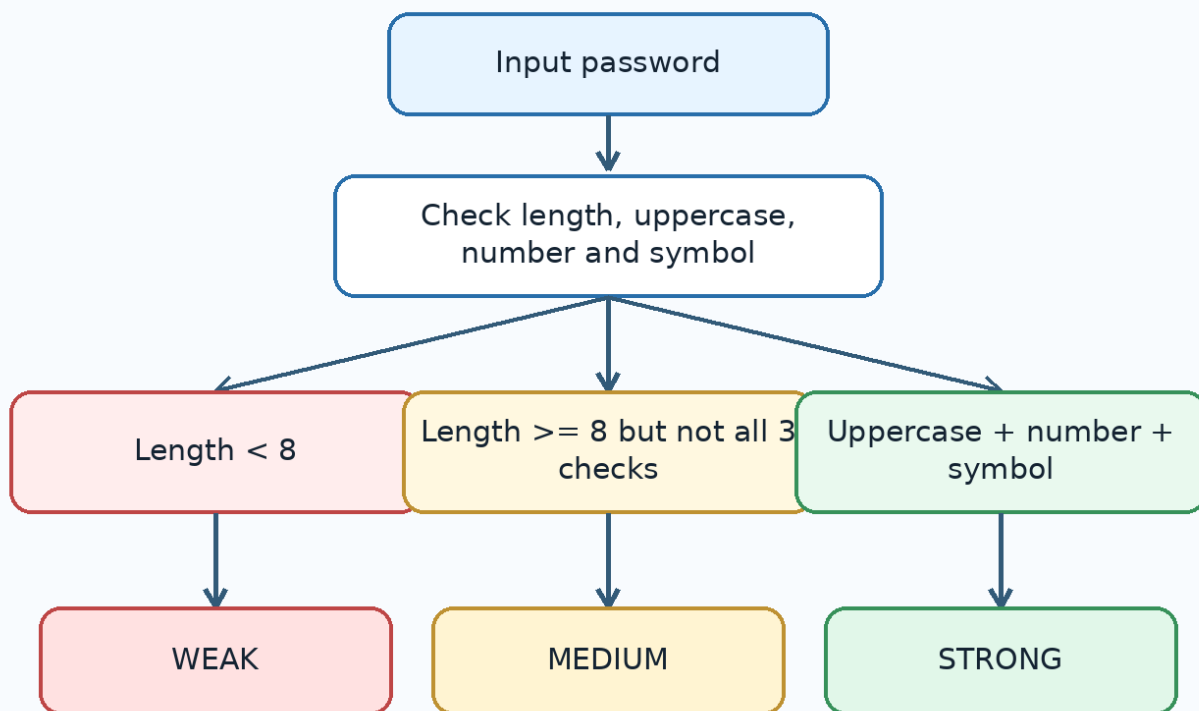
The current program uses a simple three-level classification policy:

Classification	Condition	Meaning
Weak	Password length is less than 8 characters.	The password fails the minimum-length check.
Strong	Length is at least 8 and the password contains an uppercase letter, a number, and a symbol.	All checks used by the program are satisfied.
Medium	Length is at least 8 but one or more of the uppercase, number, or symbol checks are missing.	The password passes length but does not meet every complexity check.

4. IPO Model and Algorithm

IPO means Input - Process - Output. The Password Strength Checker follows this model directly:

Input	Process	Output
A password entered by the user.	Check length, uppercase characters, digits, symbols, and apply if/elif/else conditions.	Weak, Medium, or Strong.



Decision logic used by the submitted Python program

Figure 1. Decision flow of the submitted program

5. Python Implementation

The following is the submitted Python code used for Project 1:

```
password = input("Enter your password: ")

has_uppercase = any(char.isupper() for char in password)
has_number = any(char.isdigit() for char in password)
has_symbol = any(not char.isalnum() for char in password)

if len(password) < 8:
    strength = "Weak"

elif has_uppercase and has_number and has_symbol:
    strength = "Strong"

else:
    strength = "Medium"

print("Password Strength:", strength)
```

The implementation is compact and uses Python built-ins such as `any()`, `isupper()`, `isdigit()`, and `isalnum()`. This matches the Pythonic validation approach shown in the DecodeLabs training material.

6. Code Explanation

User input: `password = input(...)` stores the password typed by the user in the variable `password`.

Uppercase check: `any(char.isupper() for char in password)` scans the characters and becomes `True` as soon as an uppercase letter is found.

Number check: `any(char.isdigit() for char in password)` checks whether at least one numeric digit exists.

Symbol check: `any(not char.isalnum() for char in password)` treats any non-alphanumeric character as a symbol.

Weak decision: If the password has fewer than 8 characters, the result is immediately `Weak`.

Strong decision: If the password is at least 8 characters long and contains uppercase, number, and symbol checks, it is `Strong`.

Medium decision: All remaining passwords of length 8 or more are classified as `Medium`.

Output: The final strength value is displayed using `print()`.

7. Test Cases and Results

The program was validated using representative passwords for each category. These examples demonstrate the exact classification rules used by the code.

Weak Example

Python Console

Enter your password: abc123

Password Strength: Weak

Reason: The password is shorter than 8 characters, so it is classified as Weak before the complexity checks can make it Strong.

Medium Example

Python Console

Enter your password: password1

Password Strength: Medium

Reason: The password is at least 8 characters long and contains a number, but it does not contain an uppercase letter and a symbol. Therefore, it is Medium.

Strong Example

Python Console

Enter your password: Secure1!

Password Strength: Strong

Reason: The password is at least 8 characters long and contains an uppercase letter, a digit, and a symbol. Therefore, it satisfies all Strong conditions in the submitted logic.

8. Requirement Validation

The following checklist confirms that the submitted implementation covers the core requirements of Project 1:

- ✓ Minimum password length is checked.
- ✓ Uppercase character presence is checked.
- ✓ Numeric digit presence is checked.
- ✓ Symbol presence is checked.
- ✓ Password is classified into Weak, Medium, or Strong.
- ✓ The final result is displayed to the user.
- ✓ The solution uses string handling and conditional logic, which are the key skills named in the task.

9. Security Notes and Limitations

This project is a learning implementation of password validation logic. The following points are useful when describing its security scope:

No password storage: The program only checks the entered text and prints a classification. It does not save the password to a file or database.

Length has highest priority: Any password shorter than 8 characters is Weak, even if it contains uppercase letters, numbers, and symbols.

Lowercase is not a separate requirement: The current task and submitted code do not independently require a lowercase letter.

Whitespace behavior: Because the symbol test is not `char.isalnum()`, spaces and other whitespace characters are also counted as symbols by the current code.

No common-password detection: A long but commonly used password may still receive Medium or Strong depending on its characters. The training material mentions checking leaked/common passwords as a possible extension, but it is not part of the current implementation.

Not a complete real-world password policy: Production password systems may use longer minimum lengths, breached-password screening, rate limiting, hashing, and multi-factor authentication. Those controls are outside this task.

Possible Future Enhancements

Enhancement	Purpose
Require lowercase letters	Add another character-variety rule.
Reject common/leaked passwords	Prevent easily guessed passwords even when they meet formatting rules.
User-friendly feedback	Tell the user which requirement is missing.
Hide typed input	Use secure input methods such as <code>getpass</code> for console applications.
Custom scoring	Assign points for length and character variety instead of only three fixed branches.

10. Conclusion

Project 1 successfully demonstrates the core security logic required for a basic Password Strength Checker. The Python program accepts a password, evaluates its length and character variety, and classifies it as Weak, Medium, or Strong. The implementation uses concise Python string methods and conditional statements, making the logic easy to understand and efficient for an introductory cybersecurity project.

The project also reinforces an important security principle: input validation should happen before sensitive information is accepted for later processing. Although the checker is intentionally simple, it provides a clear foundation for stronger password policies and more advanced authentication controls.

Project Completed - DecodeLabs Cyber Security Internship

Samaa Ahmad | Project 1 | Password Strength Checker